

ПРАКТИЧЕСКАЯ 6: МНОЖЕСТВЕННАЯ РЕГРЕССИЯ И ОЦЕНКА КАЧЕСТВА МОДЕЛИ

Ожидаемое время выполнения: 3 часа (1 час — построение моделей, 1 час — анализ метрик, 1 час — оформление отчёта).

Цель практики

Освоить построение моделей множественной регрессии, интерпретацию их результатов и визуализацию метрик качества в Python, R.

Задачи практики

1. Построить модели множественной регрессии:

- Выявить влияние нескольких факторов на целевую переменную.
- Python: использование statsmodels и sklearn для построения моделей.
- R: использование функции lm().

2. Оценить качество моделей:

- Рассчитать метрики R^2 и MSE для оценки качества модели.
- Python: инструменты sklearn.metrics.
- R: функции summary() и пакеты для анализа ошибок.

3. Визуализировать результаты:

- Построение графиков зависимости предсказанных значений от реальных.

- Сравнить удобство визуализации в Python и R.

4. Провести сравнительный анализ результатов, полученных в трёх инструментах.

Формат отчета

1. Введение:

- Цель и задачи работы.
- Краткое описание подходов к множественной регрессии и метрик качества моделей.

2. Результаты работы:

- Модели множественной регрессии:
- Код и результаты из Python и R.
- Визуализация метрик:
- Графики и таблицы сравнения R^2 и MSE между Python, R
- Интерпретация результатов:
- Какие факторы оказали наиболее значительное влияние на целевую переменную?

3. Заключение:

- Анализ преимуществ и недостатков каждого инструмента для построения и анализа регрессионных моделей.

- Рекомендации по использованию Python, R в зависимости от задач.

Дополнительные материалы к практической работе 6

Исходные данные: Таблица с множественными факторами и одной целевой переменной (например, данные о продажах, учитывающие маркетинг, цену и сезонность).

ПОШАГОВАЯ ИНСТРУКЦИЯ К ПРАКТИЧЕСКОЙ РАБОТЕ 6

ШАГ 1. ПОДГОТОВКА ДАННЫХ

1.1. Общие требования к данным

Тип данных: Вам понадобится таблица, где одна переменная будет целевой (зависимой), а несколько других переменных — факторами (независимыми признаками).

Очистка данных:

Убедитесь, что в датафрейме нет пропусков (или они корректно обработаны).

При наличии категориальных переменных, которые хотите включить в регрессию, используйте метод кодирования (One-Hot Encoding или dummy-переменные) там, где это уместно.

Проверка на мультиколлинеарность: множественная регрессия может страдать от мультиколлинеарности, когда факторы сильно коррелируют между собой. Желательно предварительно проверить корреляционную матрицу или посчитать показатель VIF (Variance Inflation Factor).

1.2. Python (statsmodels, sklearn)

Импорт библиотек:

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
```

```
import matplotlib.pyplot as plt
```

Загрузка данных:

```
df = pd.read_csv('path_to_your_file.csv') # Или через Google Colab upload  
print(df.head())
```

Проверка:

```
print(df.info())  
print(df.describe())
```

(Опционально) Кодирование категориальных переменных:

```
# Пример: если у вас есть категориальный столбец 'Category'  
df = pd.get_dummies(df, columns=['Category'], drop_first=True)
```

1.4. R (lm(), дополнительные пакеты)

Загрузка библиотек (при необходимости):

```
library(readr) # для чтения CSV  
library(ggplot2) # для визуализаций
```

Импорт данных:

```
df <- read_csv("path_to_your_file.csv")  
head(df)
```

Проверка структуры:

```
str(df)  
summary(df)
```

(Опционально) Кодирование категориальных переменных:

```
df$Category <- as.factor(df$Category)
```

ШАГ 2. ПОСТРОЕНИЕ МНОЖЕСТВЕННОЙ РЕГРЕССИИ

2.1. Теоретические аспекты

Множественная линейная регрессия: модель вида

$$Y = \beta_0 + \beta_1 \times X_1 + \beta_2 \times X_2 + \dots + \beta_n \times X_n + \epsilon$$

где Y — зависимая переменная, $X_1 \dots X_n$ — независимые факторы, β_i — коэффициенты регрессии, ϵ — ошибка.

Коэффициенты β_i показывают, на сколько в среднем изменится Y при изменении X_i на единицу (при фиксированных остальных факторах).

Мультиколлинеарность: если факторы сильно коррелируют между собой, регрессионная модель может быть нестабильной, а коэффициенты — недостоверными.

Метрики качества:

R^2 (коэффициент детерминации): доля объяснённой дисперсии зависимой переменной.

MSE (Mean Squared Error, среднеквадратическая ошибка):

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Дополнительно часто используют RMSE (корень из MSE), MAE (средняя абсолютная ошибка) и др.

2.2. Python

Построение модели с statsmodels

```
# Предположим, что 'Target' — целевая переменная
# а ['Factor1', 'Factor2', 'Factor3'] — независимые факторы

X = df[['Factor1', 'Factor2', 'Factor3']]
Y = df['Target']

# Добавляем константу в матрицу признаков
X = sm.add_constant(X)

# Строим модель
model_sm = sm.OLS(Y, X).fit()
print(model_sm.summary())
```

Построение модели с sklearn

```
model_sk = LinearRegression()
model_sk.fit(X, Y)
```

```
# Коэффициенты
print("Intercept:", model_sk.intercept_)
print("Coefficients:", model_sk.coef_)

# Предсказание
y_pred = model_sk.predict(X)
```

Расчёт метрик качества

```
r2 = r2_score(Y, y_pred)
mse = mean_squared_error(Y, y_pred)

print("R^2:", r2)
print("MSE:", mse)
```

2.3. R (lm())

Построение модели множественной регрессии

```
# Предположим, что Target – целевая переменная,
# а Factor1, Factor2, Factor3 – факторы.

model <- lm(Target ~ Factor1 + Factor2 + Factor3, data = df)
summary(model)
```

Дополнительные метрики

```
predictions <- predict(model, df)
residuals <- df$Target - predictions
mse <- mean(residuals^2)
mse

# install.packages("Metrics")
library(Metrics)
mse_val <- mse(df$Target, predictions)
r2_val <- R2(df$Target, predictions)
print(paste("MSE =", mse_val))
print(paste("R^2 =", r2_val))
```

ШАГ 3. ВИЗУАЛИЗАЦИЯ РЕЗУЛЬТАТОВ И МЕТРИК КАЧЕСТВА

3.1. Python

График «предсказанные vs. реальные значения»:

```
plt.scatter(Y, y_pred)
plt.xlabel("Реальные значения (Y)")
plt.ylabel("Предсказанные значения (Y_pred)")
plt.title("Сравнение реальных и предсказанных значений")
plt.show()
```

График остатков:

```
residuals = Y - y_pred
plt.scatter(y_pred, residuals)
plt.xlabel("Предсказанные значения (Y_pred)")
plt.ylabel("Остатки (residuals)")
plt.title("График остатков")
plt.axhline(y=0, color='red')
plt.show()
```

3.2. R

График предсказанных vs. реальных:

```
predictions <- predict(model, df)
plot(df$Target, predictions,
     xlab = "Реальные значения (Y)",
     ylab = "Предсказанные значения (Y_pred)",
     main = "Реальные vs. Предсказанные")
abline(a=0, b=1, col="red")
```

Диаграммы остатков (базовые):

```
plot(model)
```

ШАГ 4. СРАВНИТЕЛЬНЫЙ АНАЛИЗ РЕЗУЛЬТАТОВ

Сравните коэффициенты $\beta_0, \beta_1, \dots$ в Python, R.

Убедитесь, что значения примерно совпадают (за вычетом погрешности округления).

Сравните R^2 и MSE в двух инструментах.

Запишите результаты в таблицу, отметьте, если есть небольшие расхождения (чаще всего они возникают из-за разных методов округления).

Интерпретация:

Определите, какие факторы оказали наибольшее влияние на целевую переменную (обычно смотрят на величину и знак коэффициентов, а также на их р-значения).

Проанализируйте, есть ли мультиколлинеарность (например, по VIF или высокому значению корреляции между факторами).

КЛЮЧЕВЫЕ ПОЛОЖЕНИЯ ДЛЯ УСПЕШНОЙ ЗАЩИТЫ ПРАКТИЧЕСКОЙ РАБОТЫ

Убедитесь, что все коэффициенты регрессии и ключевые метрики (R^2 , MSE) выведены и интерпретированы.

Продемонстрируйте сравнение результатов между Python, R

Покажите визуализации, отражающие корректность и наглядность построенных моделей (графики остатков, реальные vs. предсказанные).

В заключении обязательно укажите, где какой инструмент наиболее удобен (например, Python — для гибкой автоматизации и расширенного анализа, R — для классических статистических методов).

Такой формат обеспечит логическую последовательность выполнения практической работы, а также позволит систематически сравнить возможности трёх инструментов при построении и оценке моделей множественной линейной регрессии.